

Communication

Axios

Axios is a promise-based HTTP client used to make HTTP requests from:

- Browser (frontend)
- Node.js (backend)

It is commonly used in React, Vue, and Node applications.

Why Use Axios?

It helps you:

- Send API requests (GET, POST, PUT, DELETE)
- Handle responses easily
- Automatically parse JSON
- Intercept requests/responses
- Handle errors cleanly

Basic Example

GET Request

```
import axios from "axios";

axios.get("https://api.example.com/users")
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.error(error);
  });
```

POST Request

```
axios.post("https://api.example.com/users", {  
  name: "Prajwal",  
  email: "test@gmail.com"  
});
```

1 Promise-based

Works with .then() and async/await.

```
const response = await axios.get("/users");
```

2 Automatic JSON parsing

No need for response.json() like fetch.

3 Interceptors (Very Powerful)

Used for:

- Adding JWT token automatically
- Logging
- Global error handling

1 Passing Headers in a Single Request

✓ GET Request with Headers

```
import axios from "axios";
```

```
axios.get("https://api.example.com/users", {  
  headers: {  
    Authorization: "Bearer YOUR_TOKEN",  
    "Content-Type": "application/json"  
  }  
});
```

Axios is a promise-based HTTP client for making API requests in browser or Node.js. It supports async/await, automatic JSON parsing, interceptors, and better error handling compared to fetch. It's widely used in frontend frameworks like React.

1 Single Request (One-Time Request)

Client sends request → Server responds → Connection closes.

Flow:

Client → Server → Response → Done

Example (Axios)

```
axios.get("/notifications");
```

Used when:

- Fetching user profile
- Submitting a form
- Getting dashboard data once

✓ Pros

- Simple
- Low server load
- Easy to scale

✗ Cons

- No real-time updates
- Client must refresh manually

2 Polling

Client repeatedly asks server for updates at fixed intervals.

Flow:

Client → Server

(wait 5 sec)

Client → Server

(wait 5 sec)

Client → Server

...

Example

```
setInterval(() => {  
  axios.get("/notifications");  
}, 5000); // every 5 seconds
```

Used when:

- Checking new notifications
- Order status updates
- Chat (basic implementation)

Types of Polling

◆ 1. Short Polling

Request every few seconds regardless of changes.

◆ 2. Long Polling

Client sends request → Server holds request until data changes → then responds → client immediately sends new request.

Better than short polling.

⚡ When to Avoid Polling?

For high-scale systems.

Example:

If 10,000 users poll every 5 seconds:

$10,000 \times (60/5) = 120,000$ requests per minute

That's heavy.

Better Alternatives (Modern Systems)

- **WebSockets**
- **Server-Sent Events (SSE)**
- Event-driven architecture

These push updates instead of constantly asking.

Single request is a one-time client-server interaction. Polling repeatedly sends requests at intervals to check for updates. Polling increases server load and network traffic, so for real-time systems we prefer WebSockets or event-driven approaches

WebSockets

WebSockets provide a **persistent, full-duplex connection** between client and server.

Meaning:

- Connection stays open
- Both client and server can send data anytime
- No repeated HTTP requests

Normal HTTP vs WebSocket

HTTP (Request–Response)

Client → Request

Server → Response

Connection closes

WebSocket

Client ⇌ Server

(continuous open connection)

How It Works Internally

1. Client sends HTTP request with Upgrade: websocket
2. Server upgrades the connection
3. TCP connection stays open
4. Messages flow both ways

After upgrade → No more HTTP overhead.

Simple Example (Node.js + ws)

`npm install ws`

Server

```
const WebSocket = require("ws");

const wss = new WebSocket.Server({ port: 8080 });

wss.on("connection", (ws) => {
  console.log("Client connected");

  ws.on("message", (message) => {
    console.log("Received:", message.toString());
    ws.send("Message received");
  });
});
```

Client (Browser)

```
const socket = new WebSocket("ws://localhost:8080");

socket.onopen = () => {
  socket.send("Hello Server");
};
```

```
socket.onmessage = (event) => {  
  console.log(event.data);  
};
```

Where WebSockets Are Used

- Chat apps
- Live trading dashboards
- Multiplayer games
- Live notifications
- Collaborative tools

Scalability Considerations

WebSockets are powerful but:

- Each user holds open connection
- Needs memory per connection
- Harder to scale horizontally

So companies use:

- Load balancers with sticky sessions
- Message brokers (Redis Pub/Sub, Kafka)
- Event-driven architecture

WebSockets provide a persistent, full-duplex connection between client and server, enabling real-time communication without repeated HTTP requests. Unlike polling, the server can push updates instantly. They're ideal for chat apps, live dashboards, and trading systems. However, they require careful handling of scalability and connection management

Webhooks

A **Webhook** is a way for one system to **automatically send real-time data to another system** when an event happens.

Instead of you asking repeatedly (polling), the other system **pushes data to you**.

Webhook = "Event-based HTTP callback"



Normal API vs Webhook

Normal API (Polling)

Your App → Payment API → Ask: "Payment done?"

Your App → Payment API → Ask again

Webhook

Payment happens

Payment API → Sends POST request → Your Server

No repeated asking.

How Webhooks Work (Step-by-Step)

1. You register a webhook URL in third-party system.
2. Event occurs (payment, signup, shipment).
3. Third-party sends HTTP POST to your endpoint.
4. Your backend processes it.
5. You return 200 OK.

Simple Express Example

```
const express = require("express");  
const app = express();
```

```
app.use(express.json());
```

```
app.post("/webhook", (req, res) => {
```

```
console.log("Webhook received:", req.body);

// Process event
res.status(200).send("OK");
});

app.listen(3000);
```

Security (Very Important)

Webhooks must be verified.

Common methods:

- Signature verification
- Secret key in header
- IP whitelisting

Example header:

x-signature: abc123xyz

You validate it before trusting the request.

When to Use Webhooks

- Payment confirmation
- GitHub push events
- Order shipping updates
- Email delivery status
- SaaS integrations

Example: **GitHub** sends webhook when:

- Someone pushes code
- PR created

- Issue opened

Challenges

- Retries (what if your server is down?)
- Idempotency (avoid double processing)
- Security validation
- Handling high event volume

A webhook is an event-driven HTTP callback where one system automatically sends data to another when an event occurs. Unlike polling, the receiving system doesn't repeatedly request updates. Webhooks are widely used in payment systems, GitHub events, and SaaS integrations. They require proper signature validation and idempotency handling for reliability.

Single Request vs Polling vs WebSockets vs Webhooks

Feature	Single Request	Polling	WebSockets	Webhooks
Communication Type	Request–Response	Repeated Request–Response	Persistent Bi-directional	Event-based HTTP callback
Connection	Closes after response	Opens & closes repeatedly	Stays open	No persistent connection
Who Initiates?	Client	Client	Both Client & Server External Server	
Real-time Capability	 No	 Delayed	 Instant	 Instant (event-based)
Server Load	Low	High	Moderate (open connections)	Low

Feature	Single Request	Polling	WebSockets	Webhooks
Network Usage	Low	High	Efficient	Efficient
Best For	Fetching data once	Periodic updates	Live apps	System integrations
Example Use Case	Fetch profile	Check order status	Chat app	Payment confirmation
Scalability	Easy	Expensive at scale	Needs connection management	Scales well

Single request is a one-time HTTP interaction. Polling repeatedly requests updates and increases server load. WebSockets maintain a persistent bi-directional connection for real-time apps. Webhooks are event-driven server-to-server callbacks triggered automatically when events occur

CDNs

CDN (Content Delivery Network) is a distributed network of servers placed across multiple geographic locations to deliver content faster to users.

Instead of your server serving everyone globally, CDN serves users from the **nearest edge location**.

Simple Idea

Without CDN:

User (India) → Server (USA) → Response (Slow)

With CDN:

User (India) → CDN Server (Mumbai) → Response (Fast)

Popular CDN Providers

- Cloudflare
 - Akamai Technologies
 - Amazon CloudFront
 - Fastly
-

How CDN Works (Step-by-Step)

1. User requests an asset (image, CSS, JS, video).
2. DNS routes request to nearest CDN edge server.
3. If file exists in cache → serve immediately.
4. If not → CDN fetches from origin server.
5. CDN caches it for future users.

What Does CDN Cache?

- Images
- Videos
- CSS / JS files
- Static HTML
- Fonts
- API responses (optional)

Benefits

-  **Faster Load Time**

Less physical distance = lower latency.

✓ **Reduced Server Load**

Your origin server handles fewer requests.

✓ **Better Scalability**

Handles traffic spikes (e.g., product launch).

✓ **DDoS Protection**

Many CDNs provide built-in security.

CDN Challenges

- Cache invalidation
- Dynamic content handling
- Stale data issues
- Cost optimization

A CDN is a globally distributed network of edge servers that cache and deliver static or cacheable content from locations closest to users, reducing latency and server load. It improves performance, scalability, and security by minimizing origin server hits and handling traffic spikes efficiently.

Proxy

A Proxy Server acts as an intermediary between client and internet.

Client → Proxy → Internet

The server does not know the real client.

◆ **Forward Proxy**

Flow

Client → Proxy → Target Server

Used on **client side**.

Purpose

- Hide client identity
- Bypass restrictions
- Content filtering
- Corporate network control

Real Example

In companies:

Employee → Corporate Proxy → Internet

Proxy blocks:

- Social media
- Adult websites
- Restricted content

Reverse Proxy

A **Reverse Proxy** sits in front of servers.

Client → Reverse Proxy → Backend Server

Client does not know which server handled request.

Flow

Client → Reverse Proxy → Server A / Server B

Used on **server side**.

Common Reverse Proxy Tools

- **NGINX**
- **HAProxy**
- **Apache HTTP Server**
- **Cloudflare**

Why Use Reverse Proxy?

Load Balancing

Distributes traffic across multiple servers.

SSL Termination

Handles HTTPS so backend stays HTTP.

Security

Hides internal server IPs.

Caching

Can cache responses.

API Gateway behavior

Forward vs Reverse Proxy

Feature	Forward Proxy	Reverse Proxy
Position	In front of client	In front of server
Hides	Client identity	Server identity
Used by	Corporate networks	Production systems
Load balancing		

In Real SaaS Architecture

User



CDN



Reverse Proxy (NGINX)



App Servers



Database

Reverse proxy is almost always used in scalable systems.

A forward proxy sits between client and internet and hides client identity, often used in corporate networks. A reverse proxy sits in front of backend servers and routes client requests to appropriate servers. Reverse proxies provide load balancing, SSL termination, caching, and improved security.

Why Use a Proxy?

We use proxies to add a control layer between client and server. Forward proxies are used for privacy and traffic control on the client side, while reverse proxies are used in production systems for load balancing, security, SSL termination, caching, and routing

Why Use a Forward Proxy?

(Proxy in front of client)

1 Hide Client Identity

Real IP is hidden.

Used for:

- Privacy
- Bypassing geo-restrictions

2 Content Filtering

Companies block:

- Social media
 - Gambling
 - Adult sites
-

Monitoring & Logging

All outgoing traffic can be logged.

Corporate example:

Employee → Proxy → Internet

Company sees everything.

Load Balancer

What Is a Load Balancer?

A **Load Balancer** distributes incoming traffic across multiple servers to ensure:

- High availability
- Scalability
- Fault tolerance
- Better performance

Instead of:

Client → Server 1 (Overloaded )

You get:

Client → Load Balancer → Server 1
 Server 2
 Server 3

Why Do We Need It?

If you have 1 server:

- It can crash
- It can overload

- It becomes bottleneck

Load balancer solves this.

How It Works (Step-by-Step)

1. Client sends request.
2. Load balancer receives it.
3. It decides which backend server should handle it.
4. Forwards request.
5. Returns response to client.

Client never knows which server responded.

Load Balancing Algorithms

1 Round Robin

Requests distributed sequentially.

Req1 → S1

Req2 → S2

Req3 → S3

Req4 → S1

2 Least Connections

Server with least active connections gets request.

3 IP Hash

Same client IP always goes to same server.

Useful for session persistence.

4 Weighted Distribution

More powerful servers get more traffic.

Additional Features

Load balancers can also:

- Perform health checks
- Remove unhealthy servers
- Handle SSL termination
- Rate limiting
- Sticky sessions

Where It Sits in Architecture

User



CDN



Load Balancer



App Server Cluster



Database

Important Concept: Single Point of Failure

If you only have one load balancer → it can fail.

Solution:

- Multiple load balancers
- DNS-level load balancing

A load balancer distributes incoming traffic across multiple backend servers to improve scalability, availability, and fault tolerance. It can use algorithms like round robin or least connections and may operate at Layer 4 or Layer 7. It also performs health checks and SSL termination in production systems.

Caching

Caching means storing frequently accessed data in a fast storage layer so future requests can be served faster.

Instead of:

Client → Server → Database (slow)

You get:

Client → Server → Cache (fast)

Why Caching Is Needed

Database calls are expensive:

- Disk I/O
- Network latency
- CPU usage

Cache reduces:

- Response time
- Server load
- Database pressure

Types of Caching

1 Client-Side Cache

Stored in browser.

Example:

- Browser caching CSS, JS
- LocalStorage

2 CDN Cache

Stored globally near users.

Example:

- Images
- Static files

Popular example: **Cloudflare**

3 Server-Side Cache (In-Memory)

Stored in fast memory like:

- **Redis**
- **Memcached**

Very fast (microseconds).

4 Database Query Cache

Database itself caches query results.

How Caching Works (Basic Flow)

Cache Miss

Request → Cache ❌ → Database → Store in Cache → Return

Cache Hit

Request → Cache ✅ → Return immediately

Common Caching Strategies

◆ **1. Cache Aside (Lazy Loading) ✅ Most Common**

1. Check cache
 2. If miss → fetch DB
 3. Store in cache
-

◆ 2. Write Through

Write to cache and DB at same time.

◆ 3. Write Back

Write to cache first → DB later (async).

TTL (Time To Live)

Data expires after certain time.

Example:

Cache user profile for 5 minutes

Cache Invalidation (Hardest Problem)

If data changes in DB → cache must update.

Common strategies:

- Time-based expiry
- Event-based invalidation
- Versioning keys

Where Caching Is Used in SaaS

- User profile data
 - Product listings
 - Dashboard metrics
 - Session storage
 - API rate limits
-

⚠ When NOT to Cache

- Highly dynamic data
- Real-time stock trading prices
- Sensitive data without proper encryption

Performance Impact

Without cache:

- 200ms–500ms

With Redis cache:

- 1ms–5ms

Huge improvement.

30-Second Interview Answer

“Caching stores frequently accessed data in a fast storage layer like Redis or CDN to reduce database load and improve response time. It works using cache hits and misses, commonly with the cache-aside pattern. Key challenges include cache invalidation and consistency management.

What Is WebRTC?

WebRTC (Web Real-Time Communication) is a technology that allows browsers and apps to communicate directly with each other in real time.

It supports:

- Video calls
- Voice calls
- Screen sharing
- Peer-to-peer data transfer

Without needing a traditional server to relay media.

Instead of:

User A → Server → User B

WebRTC does:

User A ⇄ User B (Direct Peer-to-Peer)

Server is only used for signaling.

Key Components of WebRTC

WebRTC uses 3 main things:

1 Signaling Server (Required)

Used to exchange:

- Session info
- Network details
- ICE candidates

Usually implemented using:

- WebSocket
- HTTP
- Socket.io

Signaling server does NOT carry video/audio.

STUN Server

Used to discover public IP address behind NAT.

Example public STUN server:

- stun.l.google.com

TURN Server

Used when direct peer connection fails.

In this case:

User A → TURN Server → User B

TURN relays media traffic.

How WebRTC Connection Is Established

Step-by-step:

1. User A creates offer.
2. Sends offer via signaling server.
3. User B responds with answer.
4. Both exchange ICE candidates.
5. Direct peer-to-peer connection established.

After connection → media flows directly.

Real-World Examples

- **Google Meet**
- **Zoom Video Communications**
- **Discord**
- **WhatsApp**

All use WebRTC internally (or WebRTC-based tech).

Why WebRTC Is Powerful

✓ **Low Latency**

Direct peer connection.

✓ **No Constant Server Bandwidth**

Saves infrastructure cost.

 Secure

Encrypted by default (DTLS, SRTP).

WebRTC is a browser-based technology that enables real-time peer-to-peer communication for audio, video, and data transfer. It uses a signaling server to establish connection, STUN to discover public IP, and TURN to relay traffic if direct connection fails. It's widely used in video conferencing and real-time communication apps.